PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ
ENGENHARIA DE SOFTWARE

ANA PAULA
ARTUR BERTONI
BERNARDO PLOTTEGHER
LUIZ FERNANDO

MORAH: REUSO DE SOFTWARE NA GESTÃO CONDOMINIAL

Modelo de características, MPS.BR (GRU e DRU) e Database First

CURITIBA
2026

ANA PAULA
ARTUR BERTONI
BERNARDO PLOTTEGHER
LUIZ FERNANDO

MORAH: REUSO DE SOFTWARE NA GESTÃO CONDOMINIAL

Modelo de características, MPS.BR (GRU e DRU) e Database First

Trabalho apresentado ao curso de Engenharia de Software da Pontifícia Universidade Católica do Paraná, como requisito parcial de avaliação, abordando o modelo de características, os processos de reutilização do MPS.BR e a técnica Database First aplicados ao sistema Morah.

CURITIBA
2026

SUMÁRIO

1 INTRODUÇÃO 3
2 MODELO DE CARACTERÍSTICAS (PARTE I) 4
2.1 Tipos de características e de relações 7
2.2 Restrições 7
2.3 Análise automática do FeatureIDE 8
2.4 Justificativa de cada característica 8
3 MPS.BR: GERÊNCIA E DESENVOLVIMENTO PARA REUTILIZAÇÃO (PARTE II) 12
3.1 Gerência de Reutilização (GRU) 12
3.2 Desenvolvimento para Reutilização (DRU) 13
4 DATABASE FIRST (PARTE III) 16
4.1 Ferramenta utilizada 16
4.2 Procedimento 16
4.3 Artefatos gerados 17
4.4 Adaptações dos bundles 17
4.5 Resultados 18
4.6 Roteiro da demonstração 19
4.7 Limitações 19
5 CONSIDERAÇÕES FINAIS 21
REFERÊNCIAS 22

1 INTRODUÇÃO

O Morah é um sistema de gestão de condomínios composto por um backend em Spring Boot 3.5 (Java 21, banco SQLite) e um front-end React com foco em dispositivos móveis. O contrato da API (morah-openapi.yaml, versão 2.1.0) define os módulos Login, Tela Inicial, Financeiro, Portaria, Avisos, Reservas, Minha Unidade, Encomendas, Notificações e Personalização. Seis desses módulos podem ser ativados ou desativados pelo síndico de cada condomínio. Por isso, um mesmo código atende a condomínios com necessidades diferentes, o que caracteriza o Morah como uma linha de produtos de software (POHL; BÖCKLE; VAN DER LINDEN, 2005).

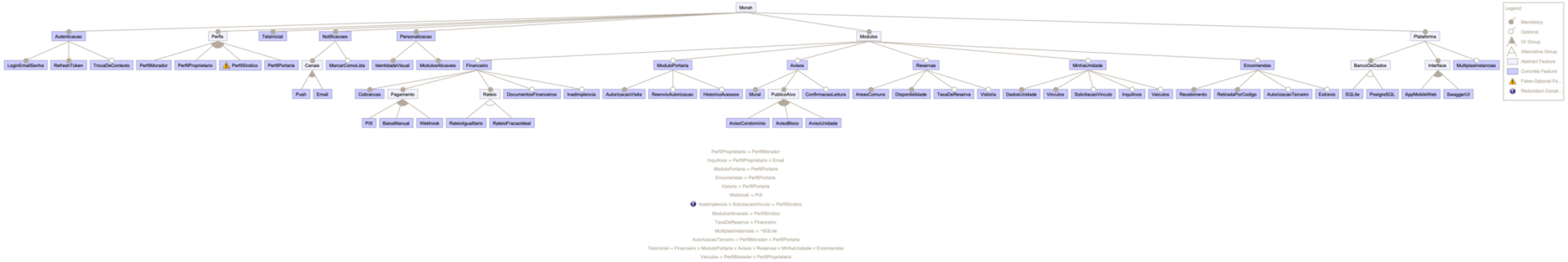
Este trabalho tem três objetivos: (i) representar a variabilidade do Morah em um modelo de características no FeatureIDE, com restrições e justificativas; (ii) classificar o sistema segundo os resultados esperados dos processos Gerência de Reutilização (GRU) e Desenvolvimento para Reutilização (DRU) do MPS.BR (SOFTEX, 2016) e propor adequações; e (iii) aplicar a técnica Database First, gerando classes, CRUD e telas automaticamente a partir do banco de dados.

Os artefatos produzidos ficam na pasta entrega do repositório morah-backend: o modelo do FeatureIDE (parte1-featureide/model.xml) e o projeto Telosys com o código gerado (parte3-telosys).

2 MODELO DE CARACTERÍSTICAS (PARTE I)

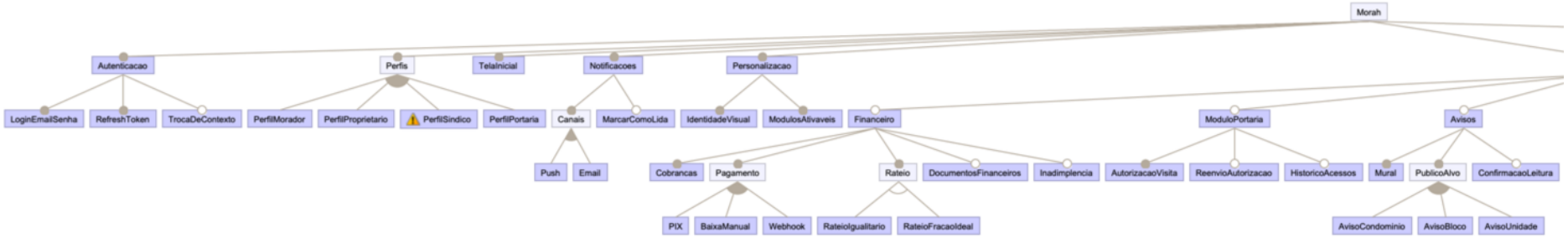
O modelo de características, proposto no método FODA (KANG et al., 1990), descreve os elementos comuns e variáveis de uma família de produtos na forma de uma árvore. Cada característica pode ser concreta ou abstrata e se liga ao pai como obrigatória, opcional ou como membro de um grupo Or ou Alternative. Restrições entre ramos diferentes da árvore completam o modelo (APEL et al., 2013). O modelo do Morah foi construído no FeatureIDE (MEINICKE et al., 2017) e tem **66 características** e **13 restrições**. As Figuras 1, 2 e 3, na página seguinte, mostram o modelo exportado do FeatureIDE.

Figura 1 – Modelo de características do Morah exportado do FeatureIDE



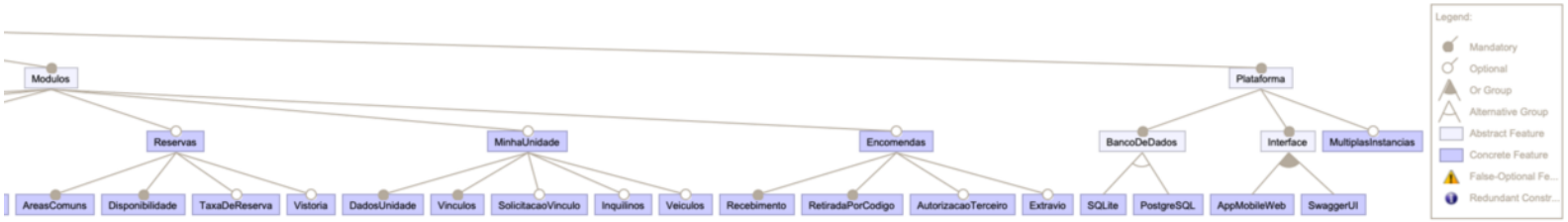
Fonte: Os autores (2026), exportado do FeatureIDE.

Figura 2 – Ampliação da Figura 1: Autenticação, Perfis, Notificações, Personalização, Financeiro, Portaria e Avisos



Fonte: Os autores (2026), exportado do FeatureIDE.

Figura 3 – Ampliação da Figura 1: Reservas, Minha Unidade, Encomendas, Plataforma e legenda



Fonte: Os autores (2026), exportado do FeatureIDE.

2.1 Tipos de características e de relações

O Quadro 1 explica cada tipo usado no modelo e mostra onde ele aparece. O modelo tem 10 características abstratas e 56 concretas. Quanto às relações, há 26 obrigatórias, 21 opcionais, 14 membros de grupos Or e 4 membros de grupos Alternative.

Quadro 1 – Tipos de características e de relações usados no modelo

Tipo	Significado	Onde foi usado e por quê
Abstract	Característica que só organiza a árvore e não corresponde a um trecho de código próprio. No FeatureIDE não aparece na configuração final.	Morah (raiz), Perfis, Modulos, Canais, Pagamento, Rateio, PublicoAlvo, Plataforma, BancoDeDados, Interface: agrupam escolhas; quem tem código são os filhos.
Concrete	Característica implementada por código (classe, endpoint, tela). Entra na configuração do produto.	LoginEmailSenha (AuthController), Veiculos (VeiculoController e VeiculosPage), RateioFracaoIdeal (classe RateioFracaoIdeal).
Mandatory	Presente em todo produto da linha sempre que o pai estiver presente.	Autenticacao, TelaInicial, Notificacoes e Personalizacao: o contrato diz que não podem ser desativados. Dentro de Financeiro, Cobrancas é obrigatória.
Optional	Pode ou não estar no produto, conforme a necessidade do condomínio.	Os 6 módulos (Financeiro, ModuloPortaria, Avisos, Reservas, MinhaUnidade, Encomendas), porque o síndico os liga e desliga; e extras como Vistoria, Veiculos e Inquilinos.
Or Group	Pelo menos um filho deve ser escolhido; vários podem coexistir.	Perfis (um condomínio pode ter qualquer combinação de papéis), Canais (Push e/ou Email), Pagamento (PIX, BaixaManual, Webhook), PublicoAlvo e Interface.
Alternative Group	Exatamente um filho deve ser escolhido.	BancoDeDados (SQLite ou PostgreSQL: uma instalação usa um banco) e Rateio (cada condomínio adota uma regra de divisão: igualitária ou por fração ideal).

Fonte: Os autores (2026).

2.2 Restrições

As restrições (cross-tree constraints) ligam características de ramos diferentes da árvore. Foram usados os quatro operadores pedidos: Implies (⇒), And (∧), Or (∨) e Not (¬). O Quadro 2 apresenta as 13 restrições com a justificativa de cada uma.

Quadro 2 – Restrições do modelo de características

Id	Restrição	Operadores	Justificativa
C1	PerfilProprietario ⇒ PerfilMorador	Implies	Proprietário é cumulativo a morador: tem tudo o que o morador tem (regra de RBAC do contrato).
C2	Inquilinos ⇒ (PerfilProprietario ∧ Email)	Implies, And	Só o proprietário cadastra inquilino, e o convite de acesso vai por e-mail.
C3	ModuloPortaria ⇒ PerfilPortaria	Implies	Sem o perfil portaria ninguém registraria visitantes.

Id	Restrição	Operadores	Justificativa
C4	Encomendas ⇒ PerfilPortaria	Implies	Quem recebe e entrega os pacotes é a portaria.
C5	Vistoria ⇒ PerfilPortaria	Implies	A vistoria da área comum é registrada pela portaria.
C6	Webhook ⇒ PIX	Implies	O webhook do gateway confirma pagamentos PIX; sem PIX ele não tem o que receber.
C7	(Inadimplencia ∨ SolicitacaoVinculo) ⇒ PerfilSindico	Implies, Or	Relatório de inadimplência e aprovação de vínculos são tarefas do síndico.
C8	ModulosAtivaveis ⇒ PerfilSindico	Implies	Só o síndico liga e desliga módulos; como ModulosAtivaveis é obrigatória, todo produto tem síndico.
C9	TaxaDeReserva ⇒ Financeiro	Implies	Cobrar a reserva gera cobrança, que só existe com o módulo financeiro.
C10	MultiplasInstancias ⇒ ¬SQLite	Implies, Not	SQLite é um arquivo local com um escritor por vez: não serve para várias instâncias.
C11	AutorizacaoTerceiro ⇒ (PerfilMorador ∧ PerfilPortaria)	Implies, And	O morador autoriza o terceiro e a portaria confere o código na retirada.
C12	TelaInicial ⇒ (Financeiro ∨ ModuloPortaria ∨ Avisos ∨ Reservas ∨ MinhaUnidade ∨ Encomendas)	Implies, Or	O painel mostra dados dos módulos: sem nenhum módulo ativo ele ficaria vazio.
C13	Veiculos ⇒ (PerfilMorador ∨ PerfilProprietario)	Implies, Or	Veículos são cadastrados pelas pessoas da unidade (morador ou proprietário).

Fonte: Os autores (2026).

2.3 Análise automática do FeatureIDE

Ao validar o modelo, o FeatureIDE apontou dois avisos, visíveis na Figura 1. O primeiro marca **PerfilSindico** como característica falsamente opcional (False-Optional): ela está num grupo Or, mas a restrição C8 (ModulosAtivaveis ⇒ PerfilSindico), somada ao fato de ModulosAtivaveis ser obrigatória, a torna presente em todo produto. O comportamento é intencional, porque todo condomínio tem um síndico, responsável por ativar os módulos.

O segundo marca a restrição **C7** ((Inadimplencia ∨ SolicitacaoVinculo) ⇒ PerfilSindico) como redundante (Redundant Constraint), já que PerfilSindico está sempre presente. Ela foi mantida porque documenta uma regra de negócio própria: o relatório de inadimplência e a aprovação de vínculos são tarefas do síndico. Se o síndico deixar de ser garantido pela C8, a regra continua valendo.

2.4 Justificativa de cada característica

O Quadro 3 lista todas as características. A coluna Relação indica como cada uma se liga ao pai, e a coluna Filhos indica o tipo de grupo que ela forma.

Quadro 3 – Características do modelo e suas justificativas

Característica	Tipo	Relação	Filhos	Justificativa
Morah	Abstract	Raiz	And	Raiz: o sistema de gestão condominial. Abstrata porque só agrupa a linha de produtos.
Autenticacao	Concrete	Mandatory	And	Todo condomínio precisa de login: sem ele nenhum dado é acessível.
LoginEmailSenha	Concrete	Mandatory	—	POST /auth/login com JWT; é a única forma de entrar no app.
RefreshToken	Concrete	Mandatory	—	Renovação rotativa do access token (POST /auth/refresh); sem ela a sessão cai a cada hora.
TrocaDeContexto	Concrete	Optional	—	Só faz sentido para quem tem vínculo com mais de um condomínio ou perfil (POST /auth/contexto).
Perfis	Abstract	Mandatory	Or	Agrupa os papéis de RBAC. Abstrata: o perfil em si não tem código, cada papel tem.
PerfilMorador	Concrete	Or Group	—	Usa os módulos da própria unidade.
PerfilProprietario	Concrete	Or Group	—	Tudo do morador e mais a gestão de inquilinos.
PerfilSindico	Concrete	Or Group	—	Administra o condomínio, os módulos e o financeiro.
PerfilPortaria	Concrete	Or Group	—	Opera visitantes, encomendas e vistorias.
TelaInicial	Concrete	Mandatory	—	Painel por perfil (GET /home/dashboard); é a porta de entrada de todo usuário.
Notificacoes	Concrete	Mandatory	And	Todos os módulos avisam as pessoas; não pode ser desativado (regra do contrato).
Canais	Abstract	Mandatory	Or	Por onde a notificação sai. Abstrata (Strategy CanalNotificacao).
Push	Concrete	Or Group	—	Notificação interna no app (CanalPush).
Email	Concrete	Or Group	—	Envio por e-mail (CanalEmail), simulado com log.
MarcarComoLida	Concrete	Optional	—	Conforto de uso: o produto funciona sem controlar a leitura.
Personalizacao	Concrete	Mandatory	And	Cada condomínio tem identidade própria; não pode ser desativada.
IdentidadeVisual	Concrete	Mandatory	—	Nome, logo e cores do condomínio (PATCH /condominio/personalizacao).
ModulosAtivaveis	Concrete	Mandatory	—	O síndico liga e desliga módulos (/condominio/modulos); é o mecanismo da variabilidade.
Modulos	Abstract	Mandatory	And	Agrupa os seis módulos ativáveis. Abstrata: não tem código próprio.
Financeiro	Concrete	Optional	And	Ativável: há condomínios que usam outra administradora.
Cobrancas	Concrete	Mandatory	—	Sem cobranças não existe módulo financeiro.
Pagamento	Abstract	Mandatory	Or	Formas de quitar a cobrança. Abstrata (Template Method ProcessamentoPagamento).
PIX	Concrete	Or Group	—	Copia e cola gerado pelo gateway simulado.

Característica	Tipo	Relação	Filhos	Justificativa
BaixaManual	Concrete	Or Group	—	Síndico registra pagamento feito fora do sistema.
Webhook	Concrete	Or Group	—	Gateway confirma o pagamento sozinho (HMAC).
Rateio	Abstract	Mandatory	Alternative	Regra que divide a taxa entre as unidades. Abstrata (Strategy EstrategiaRateio).
RateioIgualitario	Concrete	Alternative Group	—	Mesmo valor para todas as unidades.
RateioFracaoIdeal	Concrete	Alternative Group	—	Valor proporcional à fração ideal da unidade.
DocumentosFinanceiros	Concrete	Optional	—	Prestação de contas publicada; dispensável num condomínio pequeno.
Inadimplencia	Concrete	Optional	—	Relatório gerencial do síndico; não é necessário para cobrar.
ModuloPortaria	Concrete	Optional	And	Ativável: condomínio sem portaria física não usa.
AutorizacaoVisita	Concrete	Mandatory	—	Núcleo do módulo: o morador autoriza a entrada.
ReenvioAutorizacao	Concrete	Optional	—	Reenviar o pedido ao morador é uma conveniência.
HistoricoAcessos	Concrete	Optional	—	Registro de entradas e saídas; nem todo condomínio audita.
Avisos	Concrete	Optional	And	Ativável: o condomínio pode usar outro canal (ex.: mural físico).
Mural	Concrete	Mandatory	—	Lista de avisos vigentes; sem ele não há módulo.
PublicoAlvo	Abstract	Mandatory	Or	Para quem o aviso vai. Abstrata: é só a escolha do alcance.
AvisoCondominio	Concrete	Or Group	—	Aviso para todas as unidades.
AvisoBloco	Concrete	Or Group	—	Aviso para um bloco.
AvisoUnidade	Concrete	Or Group	—	Aviso para uma unidade.
ConfirmacaoLeitura	Concrete	Optional	—	Saber quem leu é útil, mas o aviso funciona sem isso.
Reservas	Concrete	Optional	And	Ativável: condomínio sem área comum não usa.
AreasComuns	Concrete	Mandatory	—	Cadastro das áreas reserváveis.
Disponibilidade	Concrete	Mandatory	—	Grade de horários; sem ela haveria reservas sobrepostas.
TaxaDeReserva	Concrete	Optional	—	Cobrar pelo uso da área é decisão do condomínio.
Vistoria	Concrete	Optional	—	Checagem de entrada e saída com registro de dano; opcional.
MinhaUnidade	Concrete	Optional	And	Ativável: o síndico pode cadastrar moradores fora do app.
DadosUnidade	Concrete	Mandatory	—	Dados da unidade do contexto ativo.
Vinculos	Concrete	Mandatory	—	Quem mora ou é dono da unidade.
SolicitacaoVinculo	Concrete	Optional	—	Pedido de novo morador com aprovação do síndico.
Inquilinos	Concrete	Optional	—	Só existe onde há locação.
Veiculos	Concrete	Optional	—	Cadastro de veículos (tabela database first); nem todo condomínio tem garagem.

Característica	Tipo	Relação	Filhos	Justificativa
Encomendas	Concrete	Optional	And	Ativável: condomínio sem recebimento de pacotes não usa.
Recebimento	Concrete	Mandatory	—	Portaria registra o pacote; é o núcleo do módulo.
RetiradaPorCodigo	Concrete	Mandatory	—	Código de 6 dígitos garante a entrega à pessoa certa.
AutorizacaoTerceiro	Concrete	Optional	—	Morador autoriza outra pessoa a retirar.
Extravio	Concrete	Optional	—	Abertura de ocorrência quando o pacote some.
Plataforma	Abstract	Mandatory	And	Decisões de infraestrutura do produto. Abstrata.
BancoDeDados	Abstract	Mandatory	Alternative	Exatamente um banco por instalação. Abstrata.
SQLite	Concrete	Alternative Group	—	Arquivo local: ideal para demonstração e condomínio pequeno.
PostgreSQL	Concrete	Alternative Group	—	Servidor de banco para produção com vários acessos simultâneos.
Interface	Abstract	Mandatory	Or	Como o usuário acessa a API. Abstrata.
AppMobileWeb	Concrete	Or Group	—	Front React mobile-first (morah-frontend).
SwaggerUI	Concrete	Or Group	—	Documentação navegável da API (springdoc).
MultiplasInstancias	Concrete	Optional	—	Escalar horizontalmente; só necessário em condomínios grandes ou SaaS.

Fonte: Os autores (2026).

3 MPS.BR: GERÊNCIA E DESENVOLVIMENTO PARA REUTILIZAÇÃO (PARTE II)

O Guia Geral MPS de Software (SOFTEX, 2016) define os processos Gerência de Reutilização (GRU), do nível E, e Desenvolvimento para Reutilização (DRU), do nível C. Para cada resultado esperado, o Morah foi classificado em verde (de acordo com o resultado esperado), amarelo (identificação parcial) ou vermelho (não encontrado). A avaliação considera os repositórios morah-backend e morah-frontend no estado atual.

3.1 Gerência de Reutilização (GRU)

O propósito da GRU é gerenciar o ciclo de vida dos ativos reutilizáveis (SOFTEX, 2016, p. 36). O Quadro 4 apresenta a classificação dos resultados GRU 1 a GRU 5.

Quadro 4 – Classificação do Morah nos resultados esperados da GRU

Resultado	O que o guia espera	Classificação	Justificativa no Morah
GRU 1	Estratégia de gerenciamento de ativos documentada (definição de ativo reutilizável, critérios de aceitação, certificação, classificação, descontinuidade e avaliação).	**Amarelo**	Há documentação dos ativos reutilizáveis: a seção "Design patterns" do README descreve cada um dos 8 usos de padrões e o problema que resolve, e docs/refatoracao-reuso.md registra o critério usado na refatoração (trecho repetido vira helper único). Falta uma estratégia formal: não há definição oficial de "ativo reutilizável", nem critérios de aceitação, certificação, classificação, descontinuidade ou avaliação.
GRU 2	Mecanismo de armazenamento e recuperação de ativos implantado.	**Amarelo**	Os ativos ficam versionados no Git/GitHub (morah-backend e morah-frontend) e organizados por tipo: pacote pattern.<tipo>.<uso>, pasta componentes/ no front, bundles de templates em TelosysTools/templates. A recuperação, porém, é feita navegando no código: não existe catálogo, busca por metadados nem publicação como artefato (pacote Maven ou npm).
GRU 3	Dados de utilização dos ativos reutilizáveis registrados.	**Amarelo**	O README registra onde cada padrão é usado (ex.: BaseCrudController usado por AvisoController, ReservaController e EncomendaController; NotificacaoTemplate com 6 subclasses). É um registro estático, escrito à mão: não há contagem de uso, histórico de adoção nem atualização automática.
GRU 4	Ativos mantidos periodicamente segundo os critérios definidos, com modificações controladas ao longo do ciclo de vida.	**Amarelo**	As modificações são controladas: commits convencionais, branches e PRs por funcionalidade, e testes unitários dedicados para cada padrão (7 classes em src/test/.../pattern). A refatoração de reuso foi uma manutenção explícita. Falta a periodicidade e um critério definido de manutenção (revisão programada dos ativos).

Resultado	O que o guia espera	Classificação	Justificativa no Morah
GRU 5	Usuários dos ativos notificados sobre problemas, modificações, novas versões e descontinuidade.	**Vermelho**	Não há mecanismo de notificação: nem changelog, nem release notes, nem aviso de descontinuidade. O único indício é a versão do contrato OpenAPI (2.0.0 → 2.1.0), que sinaliza ao front uma mudança na API, mas ninguém é avisado ativamente.

Fonte: Os autores (2026).

Como nenhum resultado está totalmente de acordo, o Quadro 5 propõe como adequar o protótipo a cada nível da GRU.

Quadro 5 – Proposta de adequação aos resultados GRU 1 a GRU 5

Resultado	Como adequar o protótipo	Evidência esperada
GRU 1	Escrever docs/estrategia-reuso.md com: o que é ativo reutilizável no Morah (padrões, helpers, componentes de UI, bundles Telosys, contrato OpenAPI); critério de aceitação (usado em 2 ou mais pontos, com teste e Javadoc/JSDoc); certificação (PR revisado por outro membro e CI verde); classificação (domínio, tipo, maturidade); descontinuidade (anotação @Deprecated + prazo de uma versão) e avaliação semestral.	Documento de estratégia aprovado pela equipe.
GRU 2	Publicar os ativos estáveis como artefatos: um módulo Maven morah-commons (BaseCrudController, templates e strategies) e um pacote npm @morah/ui (Campo, Botao, Cartao, Estados), num GitHub Packages. Manter um CATALOGO.md com nome, descrição, versão, dono e link de cada ativo.	Repositório de pacotes e catálogo com busca.
GRU 3	Registrar o uso: cada projeto declara a dependência de morah-commons e @morah/ui na versão usada, e um script (ex.: grep/ArchUnit no CI) gera a tabela "ativo → onde é usado" a cada build, anexada ao catálogo.	Relatório de uso gerado automaticamente.
GRU 4	Revisão trimestral dos ativos com checklist (testes, cobertura, dependências, uso real); versionamento semântico de cada ativo; toda mudança via PR com label "ativo-reutilizavel".	Atas de revisão e histórico de versões.
GRU 5	CHANGELOG.md por ativo (formato Keep a Changelog), GitHub Releases com release notes, e GitHub Issues com label "ativo" para problemas. Quem depende do ativo assina o repositório (watch) e recebe a notificação; descontinuidade anunciada uma versão antes.	Changelog, releases e issues rastreáveis.

Fonte: Os autores (2026).

3.2 Desenvolvimento para Reutilização (DRU)

O propósito do DRU é identificar oportunidades de reutilização sistemática e, se possível, estabelecer um programa de reutilização para desenvolver ativos a partir de engenharia de domínio (SOFTEX, 2016, p. 42). O Quadro 6 apresenta a classificação dos resultados DRU 1 a DRU 9.

Quadro 6 – Classificação do Morah nos resultados esperados do DRU

Resultado	O que o guia espera	Classificação	Justificativa no Morah
DRU 1	Domínios de aplicação com oportunidades de reuso identificados.	**Amarelo**	O domínio (gestão condominial) está dividido em 6 módulos ativáveis mais login, tela inicial, notificações e personalização, e a refatoração identificou oportunidades concretas de reuso (paginação, busca de unidade, ocorrência, vínculo). A identificação foi pontual e feita no código, sem estudo dos domínios.
DRU 2	Capacidade de reutilização sistemática avaliada e ações corretivas tomadas.	**Amarelo**	Houve avaliação e correção: a duplicação nos services foi medida (17 blocos repetidos) e corrigida com Extract Method. Foi uma ação única, não uma avaliação periódica da capacidade de reuso da equipe.
DRU 3	Programa de reutilização planejado (propósitos, escopo, metas e objetivos).	**Vermelho**	Não existe programa de reutilização planejado: o reuso acontece por iniciativa de quem desenvolve.
DRU 4	Programa de reutilização implantado, monitorado e avaliado.	**Vermelho**	Sem programa (DRU 3), não há implantação, monitoramento nem avaliação.
DRU 5	Propostas de reutilização avaliadas para garantir adequação à aplicação alvo.	**Amarelo**	Cada padrão de projeto tem justificativa documentada ("problema que resolve"), e na Parte III o bundle oficial do Telosys foi avaliado, mostrou-se inadequado (Spring Boot 2.7, PostgreSQL, ModelMapper) e foi adaptado. A avaliação não segue critérios definidos.
DRU 6	Formas de representação para modelos de domínio e arquiteturas de domínio selecionadas.	**Verde**	As formas de representação estão escolhidas e em uso: modelo de características no FeatureIDE (Parte I) para o domínio; diagramas Mermaid de classes, entidade-relacionamento e sequência em docs/documentacao.md para a arquitetura; e OpenAPI 3.2 para as interfaces.
DRU 7	Modelo de domínio desenvolvido e mantido, com limites e relações com outros domínios, capturando características comuns, variantes, opcionais e obrigatórias.	**Verde**	O modelo de características da Parte I captura as obrigatórias (Autenticacao, TelaInicial, Notificacoes...), opcionais (os 6 módulos), variantes (grupos Or e Alternative) e 13 restrições entre elas. Os limites com outros domínios aparecem em Plataforma (banco, interface) e nos canais externos (e-mail, gateway PIX). Ele é mantido como código (gerar_modelo.py) no repositório.
DRU 8	Arquitetura de domínio que descreve uma família de aplicações, desenvolvida e mantida.	**Amarelo**	O código já implementa uma família de produtos: um único backend que liga e desliga módulos por condomínio (ModuloAtivoInterceptor, CacheModulosAtivos, /condominio/modulos) e seleciona comportamento por estratégia (rateio, canal, dashboard). A arquitetura está documentada, mas não é descrita como arquitetura de linha de produtos, com os pontos de variação ligados às características do modelo.
DRU 9	Ativos do domínio especificados, adquiridos ou desenvolvidos, e mantidos.	**Amarelo**	Há ativos desenvolvidos (BaseCrudController, NotificacaoTemplate, ProcessamentoPagamentoTemplate, strategies, componentes de UI, bundle morah-telas-crud) e adquiridos (bundle oficial do Telosys, Spring, TanStack Query), mantidos com testes e Git. Falta a especificação formal de cada ativo (interface, pré-condições, exemplo de uso) ligada ao modelo de domínio.

Fonte: Os autores (2026).

O Quadro 7 propõe a adequação aos resultados DRU 6, 7, 8 e 9, e resume as ações para os resultados DRU 1 a 5.

Quadro 7 – Proposta de adequação aos resultados DRU 6 a DRU 9

Resultado	Como adequar o protótipo	Evidência esperada
DRU 6	Já atendido. Para consolidar: registrar a decisão num ADR (docs/adr/001-representacao-dominio.md), definindo FeatureIDE para o modelo de domínio, C4 + Mermaid para a arquitetura e OpenAPI para as interfaces, com o motivo de cada escolha.	ADR aprovado.
DRU 7	Já atendido. Para manter: gerar o model.xml no CI a partir do gerar_modelo.py e falhar o build se uma característica concreta não tiver código ligado a ela (ex.: anotação @Caracteristica("Veiculos") nas classes); revisar o modelo a cada novo módulo.	Modelo sincronizado com o código.
DRU 8	Descrever a arquitetura de linha de produtos: um diagrama C4 com o núcleo comum (login, notificações, personalização, tela inicial) e os pontos de variação; uma tabela "característica → ponto de variação" (ex.: Financeiro → ModuloAtivoInterceptor + FinanceiroController; Rateio → EstrategiaRateio; Canais → CanalNotificacao); e uma configuração por produto (ex.: perfil Spring "condominio-pequeno" com SQLite e sem Financeiro). Manter o documento junto com o código.	Documento de arquitetura da família e configurações por produto.
DRU 9	Especificar cada ativo do domínio num cabeçalho padrão (propósito, característica do modelo que realiza, interface, exemplo de uso e testes) e publicá-los em morah-commons e @morah/ui (ver GRU 2). Novos ativos entram por PR com a especificação e são mantidos pelas regras de GRU 4.	Ativos especificados, versionados e publicados.
DRU 1 a 5	Fazer uma análise de domínio comparando o Morah com outros sistemas condominiais (DRU 1); medir duplicação no CI (ex.: PMD CPD) como indicador de capacidade de reuso (DRU 2); planejar um programa de reuso com meta, por exemplo, "50% dos controllers usando BaseCrudController no próximo semestre" (DRU 3), acompanhado na revisão trimestral (DRU 4); e avaliar cada proposta de reuso com um checklist de adequação (DRU 5).	Programa de reuso com metas e indicadores.

Fonte: Os autores (2026).

4 DATABASE FIRST (PARTE III)

Na abordagem Database First, o banco de dados é a fonte da verdade: as tabelas são modeladas primeiro e o código da aplicação é gerado ou mapeado a partir delas. É o caminho inverso do Code First, em que as classes são escritas primeiro e o framework de persistência cria as tabelas. No Morah, a tabela veiculo já havia sido criada em SQL (arquivo schema.sql) e mapeada para a entidade JPA. Nesta parte, o objetivo é gerar classes, CRUD e telas de forma automática a partir de todo o banco.

4.1 Ferramenta utilizada

A ferramenta escolhida foi o Telosys 4.3 (TELOSYS, 2025a), um gerador de código gratuito e de código aberto. Ele lê os metadados do banco por JDBC, cria um modelo com um arquivo .entity por tabela e aplica bundles de templates Velocity para gerar o código. Foi escolhido por cobrir todo o pedido: classes a partir do banco, CRUD completo e, com um bundle próprio, as telas. O banco de origem foi o próprio SQLite do Morah (morah.db), com 27 tabelas.

4.2 Procedimento

A geração foi feita pela linha de comando do Telosys, na seguinte sequência:

a) Inicialização do projeto com `init`, que cria a pasta TelosysTools com a configuração.
b) Configuração do banco em TelosysTools/databases.yaml (tipo SQLITE e URL `jdbc:sqlite:../../morah.db`), com o driver sqlite-jdbc em TelosysTools/lib.
c) Teste da conexão com `cdb morah -t`, que lista as 27 tabelas.
d) Criação do modelo a partir do banco com `nm morah morah`: 27 entidades com tipos, tamanhos e obrigatoriedade lidos das colunas.
e) Instalação do bundle oficial de CRUD com `ib java-rest-springboot-jpa-basic` (TELOSYS, 2025b).
f) Geração do backend com `b java-rest-springboot3-sqlite` e `gen * * -y`: 135 arquivos.
g) Geração das telas com `b morah-telas-crud` e `gen * * -y`: 29 arquivos.
h) Execução com `mvn spring-boot:run` e acesso em http://localhost:8081.

A entidade criada pelo Telosys para a tabela veiculo mostra que o modelo vem do banco: nomes, tipos, tamanhos e restrições de NOT NULL são os das colunas.

```
@DbTable(veiculo)
Veiculo {
  // attributes
  id : int { @Id @DbName(id) @DbType(INTEGER) };
  unidadeId : int { @DbName(unidade_id) @DbType(INTEGER) @NotNull @NotEmpty };
  placa : string { @DbName(placa) @DbType("VARCHAR(7)") @NotNull @NotEmpty @Size(7) @MaxLen(7) };
  modelo : string { @DbName(modelo) @DbType("VARCHAR(60)") @NotNull @NotEmpty @Size(60) @MaxLen(60) };
  cor : string { @DbName(cor) @DbType("VARCHAR(30)") @Size(30) @MaxLen(30) };
  tipo : string { @DbName(tipo) @DbType("VARCHAR(10)") @NotNull @NotEmpty @Size(10) @MaxLen(10) };
  // links
}
```

4.3 Artefatos gerados

Para cada tabela com chave primária são gerados os arquivos do Quadro 8. Também são gerados o pom.xml, o application.yml e as classes de inicialização do Spring Boot, de modo que o resultado é uma aplicação completa.

Quadro 8 – Arquivos gerados para a tabela veiculo

Arquivo	Bundle	Conteúdo
Veiculo.java	java-rest-springboot3-sqlite	Entidade JPA com @Table e @Column iguais às colunas do banco
VeiculoRepository.java	java-rest-springboot3-sqlite	Repositório Spring Data (JpaRepository)
VeiculoDTO.java	java-rest-springboot3-sqlite	Objeto de transferência usado no JSON
VeiculoService.java	java-rest-springboot3-sqlite	Regras de CRUD: listar, buscar, criar (409 se já existe), salvar e excluir
VeiculoRestController.java	java-rest-springboot3-sqlite	Endpoints GET, POST, PUT, PATCH e DELETE em /api/veiculo
telas/veiculo.html	morah-telas-crud	Tela com formulário e tabela para listar, cadastrar, editar e excluir
index.html, crud.js e crud.css	morah-telas-crud	Gerados uma vez: índice das telas, script genérico de CRUD e estilos

Fonte: Os autores (2026).

4.4 Adaptações dos bundles

O bundle oficial gera um projeto Spring Boot 2.7 com PostgreSQL. Para gerar código compatível com as versões usadas no Morah, ele foi copiado como java-rest-springboot3-sqlite e ajustado nos seguintes pontos:

- Spring Boot 2.7 para 3.5.16 e Java 17 para 21, com javax.persistence substituído por jakarta.persistence.
- Driver do PostgreSQL substituído pelo do SQLite (sqlite-jdbc e hibernate-community-dialects).
- ModelMapper substituído por BeanUtils.copyProperties, do próprio Spring, na classe GenericService.

- PagingAndSortingRepository substituído por JpaRepository, pois no Spring Data 3 ele não herda mais do CrudRepository.
- Tabelas sem chave primária (vistoria_fotos_url, uma coleção do Hibernate) são ignoradas em vez de interromper a geração.

Como nenhum bundle oficial do Telosys gera telas para Java, foi criado o bundle morah-telas-crud. Ele gera uma página por tabela, que apenas declara os campos lidos do modelo, e um único script genérico de CRUD (crud.js), reutilizado por todas as páginas.

4.5 Resultados

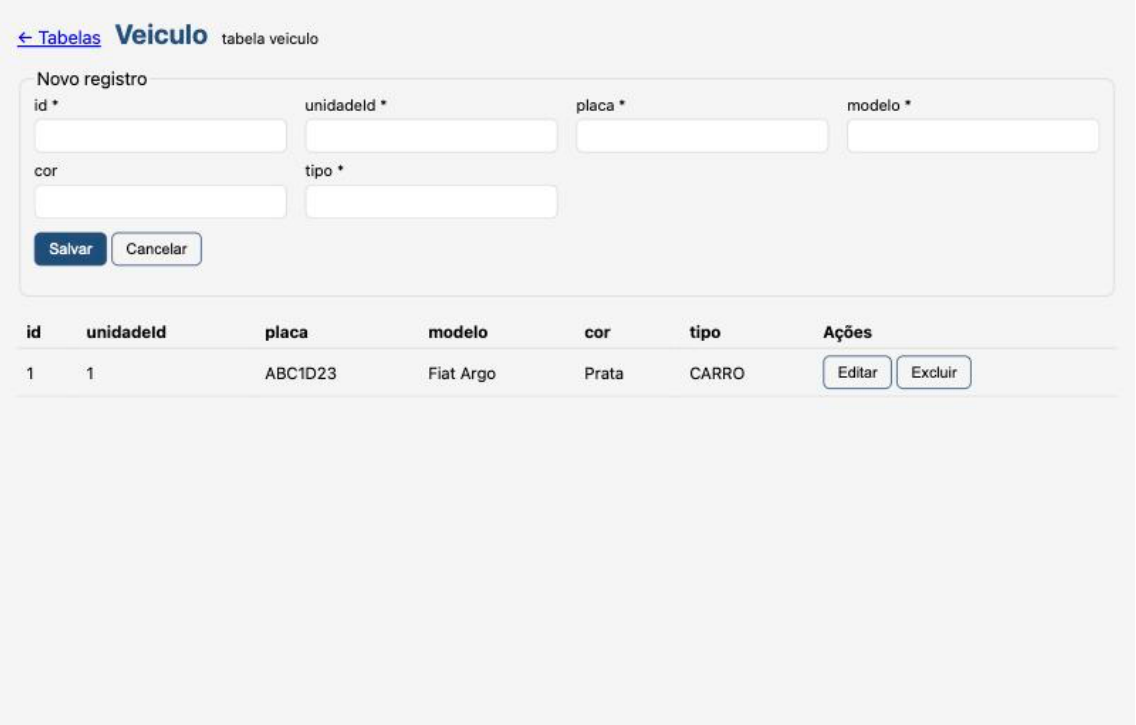
As Figuras 4 e 5 mostram a aplicação gerada em execução. O CRUD foi testado pela tela da tabela veiculo: cadastro (resposta HTTP 201), edição (200) e exclusão (204). A aplicação gerada usa uma cópia do banco (morah-demo.db), para que a demonstração não altere os dados do backend.

Figura 4 – Índice gerado, com uma tela de CRUD para cada tabela com chave primária



Fonte: Os autores (2026).

Figura 5 – Tela de CRUD gerada para a tabela veiculo, com os dados do banco



Fonte: Os autores (2026).

4.6 Roteiro da demonstração

O roteiro completo está em entrega/parte3-telosys/README.md. Em resumo, quem for demonstrar deve:

i) Mostrar a tabela no banco com sqlite3 morah.db ".schema veiculo".

j) Apagar o código gerado (rm -rf src pom.xml target), para provar que nada é escrito à mão.

k) Na CLI do Telosys, executar cdb morah -t, dm morah, nm morah morah, b java-rest-springboot3-sqlite, gen * * -y, b morah-telas-crud e gen * * -y.

l) Copiar o banco para morah-demo.db, executar mvn spring-boot:run e abrir http://localhost:8081.

m) Na tela Veiculo, listar, cadastrar, editar e excluir um registro.

n) Opcionalmente, adicionar uma coluna ao banco (por exemplo, ano), gerar o código de novo e mostrar o novo campo na tela.

4.7 Limitações

- O driver do SQLite não informa as chaves estrangeiras ao Telosys, então os relacionamentos aparecem como colunas simples (por exemplo,

unidadeId) em vez de associações @ManyToOne. Com um banco PostgreSQL, o mesmo processo traria os relacionamentos.

- O SQLite também não marca o id como autoincremento, por isso a tela pede o id no cadastro.
- O PATCH gerado pelo bundle oficial é apenas um esqueleto; as telas usam PUT, que grava o registro inteiro.

5 CONSIDERAÇÕES FINAIS

O modelo de características mostrou que a variabilidade do Morah já está presente no código: os seis módulos ativáveis, as estratégias de rateio e de canal de notificação e as escolhas de plataforma formam uma família de produtos. A análise automática do FeatureIDE ajudou a validar o modelo e a tornar explícitas regras implícitas, como a presença obrigatória do síndico.

Na avaliação do MPS.BR, o Morah tem bons indícios de reuso (padrões de projeto documentados e testados, versionamento e um modelo de domínio), mas ainda não gerencia os ativos de forma sistemática. As propostas apresentadas indicam o caminho: estratégia documentada, catálogo e publicação dos ativos, registro de uso, notificação de versões e uma arquitetura de linha de produtos ligada ao modelo de características.

Por fim, o Telosys gerou a partir do banco, sem código escrito à mão, um backend REST completo e as telas de CRUD para 26 tabelas. Isso demonstra na prática a técnica Database First e o reuso de templates de geração de código.

REFERÊNCIAS

APEL, S. et al. **Feature-oriented software product lines**: concepts and implementation. Berlin: Springer, 2013.

KANG, K. C. et al. **Feature-oriented domain analysis (FODA) feasibility study**. Pittsburgh: Software Engineering Institute, Carnegie Mellon University, 1990. (Technical Report CMU/SEI-90-TR-21).

MEINICKE, J. et al. **Mastering software variability with FeatureIDE**. Cham: Springer, 2017.

POHL, K.; BÖCKLE, G.; VAN DER LINDEN, F. **Software product line engineering**: foundations, principles and techniques. Berlin: Springer, 2005.

SOFTEX. **MPS.BR – Melhoria de Processo do Software Brasileiro**: guia geral MPS de software. [S. l.]: SOFTEX, jan. 2016.

TELOSYS. **Telosys documentation**. 2025a. Disponível em: https://doc.telosys.org/. Acesso em: 13 nov. 2026.

TELOSYS. **java-rest-springboot-jpa-basic**: Java basic REST CRUD backend with Spring Boot and Spring Data JPA. 2025b. Disponível em: https://github.com/telosys-templates/java-rest-springboot-jpa-basic. Acesso em: 13 nov. 2026.